MODULE 11 · SECTION 2 - INGESTING AND PROCESSING THE DATA

Pub/Sub: Delivery, Replay and Failure

Subscriptions, ack deadlines, replay, dead letters, exactly-once, ordering, and when Kafka fits better.

SECTION 2 of 5 EXAM WEIGHT 25% MODULE 11 of 45

admin@sthithapragna.com

Pub/Sub delivers every message **at least once**. Your design decides what happens the second time.

admin@sthithapragna.com

From meter to subscriber

01

Publish

The head-end publishes each reading to meter-readings.

02

Store

The topic stores it, checks its schema, and fans out.

03

One copy each

Every subscription gets its own copy of every message.

04

Deliver

A subscriber receives it, and must ack before the deadline.

05

Or again

No ack in time, or a nack, and it is delivered again.

THE PATTERN

Subscriptions are independent. A slow archive never holds up the live pipeline.

Four ways to receive

TYPE	HOW IT WORKS	PICK IT WHEN
Pull	Your client asks for messages	High throughput, Dataflow, exactly-once
Push	Pub/Sub POSTs to your HTTPS endpoint	Serverless receivers, such as Cloud Run
BigQuery	Writes straight into a table	No processing needed before storing
Cloud Storage	Writes batches of files to a bucket	Cheap raw archives

THE SIGNAL

No transformation, straight to a table: a BigQuery subscription, not a Dataflow job.

Sign for it in time

- Default **10 seconds**. Configurable from **10 to 600 seconds**.
- Client libraries **extend the lease** while a message is still being worked on.
- Miss the deadline and the message is **delivered again**, even if you finish later.
- A **negative ack** asks for redelivery straight away. Useful for transient errors.

> THE SIGNAL
>
> Rising duplicates with a slow subscriber usually means missed deadlines, not a chatty publisher.

How long does Pub/Sub remember?

SETTING	DEFAULT	RANGE
Subscription retention	7 days	10 minutes to 31 days
Retain acked messages	Off	Keeps acked messages for replay
Topic retention	Off	10 minutes to 31 days
Snapshot lifetime	Up to 7 days	Less if the backlog is old
Subscription expiry	31 days idle	1 day up, or never

CURRENCY CHECK

Older material says subscriptions keep messages for 7 days at most. The maximum is now 31.

admin@sthithapragna.com

Seek to a time, or to a snapshot?

SEEK TO A TIME

Rewind the clock

- Messages after the time become unacked
- Needs retained messages
- Recover from any surprise
- A future time purges a backlog

SEEK TO A SNAPSHOT

Rewind to a bookmark

- Saved ack state at one moment
- No retention settings needed
- Take one before every deploy
- Lasts up to 7 days

THE TIEBREAKER

Planned risk, like a deployment: snapshot first. Unplanned trouble: seek to a time, if retention is on.

A safe deployment at Larkspur

```
1  # 1. Before deploying new subscriber code, bookmark the ack state
2  gcloud pubsub snapshots create pre-deploy-1009 \
3      --subscription=readings-dataflow
4
5  # 2. The new code has a bug: rewind to the bookmark
6  gcloud pubsub subscriptions seek readings-dataflow \
7      --snapshot=pre-deploy-1009
8
9  # 3. No snapshot? Rewind to a time, if acked messages are retained
10 gcloud pubsub subscriptions seek readings-dataflow \
11     --time=2026-10-09T12:00:00Z
```

`2-3` Seconds to run, and it saves a bad deploy.

`6-7` Everything since the bookmark is delivered again.

`10-11` Works only with retention switched on beforehand.

REMEMBER

Replay means duplicates by design. Idempotent writes make replay safe.

Retries and dead letters

- Retry policy: **immediately** by default, or **exponential backoff** from 10 to 600 seconds.
- A **dead-letter topic** takes a message after **5 to 100** attempts. Default 5.
- The **Pub/Sub service agent** needs Publisher on the dead-letter topic, and Subscriber on the subscription.
- Forwarding is **best effort**, so watch the dead-letter topic with an alert.

> THE PATTERN
>
> A poison message retried forever blocks good work. Back off, then set it aside.

Exactly-once delivery, precisely

- **Pull subscriptions only**, including streaming pull. Not push, not BigQuery, not Storage.
- Subscribers must connect in **the same region**.
- No redelivery after a **successful ack**, and none while a message is out.
- **Publisher retries can still create copies**. Latency is noticeably higher.

> THE TRAP
>
> Exactly-once delivery is not exactly-once data. Still deduplicate on a business key.

Ordering keys

- Messages with the **same key**, published in the **same region**, arrive in order.
- Each key is limited to **1 MBps** of publishing.
- One redelivery **replays every later message** for that key, even acked ones.
- Often you can **sort by event time** instead, and skip ordering entirely.

> THE TIEBREAKER
>
> Ask whether order matters, or only the final answer. Timestamps and idempotent writes are cheaper.

Filters, schemas and transforms

- **Filters** select on **attributes only**, and cannot be changed after creation.
- **Schemas** in Avro or Protocol Buffer. A message that does not match is **not published**.
- **Single message transforms**: a JavaScript function or AI inference, on a topic or subscription.
- Transforms handle **one message at a time**. Aggregation still needs Dataflow.

> THE SIGNAL
>
> 'Reject malformed messages at the door' is a topic schema. 'Mask one field' can be a transform.

Pub/Sub or Managed Service for Apache Kafka?

PUB/SUB

Serverless messaging

- No partitions or clusters to size
- Up to 31 days of retention
- Pay for what you use
- Dead-letter topics built in

MANAGED KAFKA

Kafka, run for you

- Clusters and partitions to plan
- Indefinite retention, event sourcing
- Kafka API: existing apps move as is
- Same platform on-premises too

THE TIEBREAKER

Existing Kafka apps, minimal change: Managed Kafka. Pub/Sub Lite was turned down on 18 March 2026.

admin@sthithapragna.com

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Duplicates rise with a slow subscriber	A longer ack deadline
Reprocess after a bad deploy	Seek to a snapshot or a time
No processing, straight to a table	BigQuery subscription
Poison messages keep failing	Backoff and a dead-letter topic
Never process twice	Exactly-once on pull, plus dedupe
Existing Kafka apps	Managed Service for Apache Kafka

THE PATTERN

At least once is the default. Design every subscriber to survive the second delivery.

> A Larkspur subscriber calls a slow tariff service for every reading, taking about 30 seconds per message. Its custom client never extends the ack deadline, which is still the default.
>
> Duplicate readings downstream have risen sharply, but the head-end publishes each reading only once.

What should the team change first?

A Turn on message ordering, so that messages arrive one at a time

B Attach a dead-letter topic, with five delivery attempts

C Raise the ack deadline above the processing time

D Switch the subscription from pull delivery to push delivery

◆ PAUSE AND DECIDE

What should the team change first?

A. Message ordering

Ordering does not stop redelivery. It can make it worse.

B. A dead-letter topic

Good messages would be set aside just for being slow.

C. Longer deadline

30 seconds of work outlasts a 10-second deadline. Fix that, and the copies stop.

D. Push delivery

The endpoint would still be slow, and still miss its deadline.

THE SIGNAL

Slow work plus the default deadline means redelivery. The duplicates are self-inflicted.

> A new version of Larkspur's subscriber went live at 09:00. Until noon it acknowledged every reading but wrote nothing, because of a bug.
>
> Nobody took a snapshot. The subscription retains acknowledged messages for 7 days, and the bug is now fixed.

How should the team recover the three lost hours?

A Ask the head-end team to republish every reading sent since 09:00

B Create a snapshot of the subscription now, then seek to that snapshot

C Create a new subscription, which will receive the last three hours

D Seek the subscription to a time just before the 09:00 deployment

◆ PAUSE AND DECIDE

How should the team recover the three lost hours?

A **Republish from the head-end**

Slow, manual, and it depends on a system that may not keep them.

B **Snapshot now, then seek**

A snapshot taken now bookmarks the present, after the loss.

C **A new subscription**

New subscriptions start from now. They do not see older messages.

D **Seek to a time**

Retained acked messages after 09:00 become unacked, and are delivered again.

THE SIGNAL

No snapshot, but acked messages retained: seek to a time before the problem began.

> Larkspur wants every raw reading copied into raw.meter_readings exactly as published, with no transformation. The topic already has an Avro schema.
>
> The table accepts duplicates, and the team wants the least code and infrastructure possible.

How should the readings reach the table?

A. A Dataflow streaming job that reads the topic and writes each row

B. A push subscription to a Cloud Run service that inserts rows

C. A BigQuery subscription that uses the topic's Avro schema

D. A Cloud Storage subscription, followed by an hourly load job

◆ PAUSE AND DECIDE

How should the readings reach the table?

A **A Dataflow job**

It works, but it is a pipeline to run for no transformation at all.

B **Push to Cloud Run**

A service to write, deploy and scale, just to insert rows.

C **BigQuery subscription**

Straight into the table, mapped by the schema, with no code.

D **Storage, then load jobs**

Two steps, an hour of delay, and a scheduler to manage.

THE SIGNAL

No processing, straight to a table, least code: a BigQuery subscription.

> A few malformed readings make Larkspur's subscriber throw an error every time. Pub/Sub redelivers them immediately, again and again, which wastes capacity during the evening surge.
>
> The team wants bad messages set aside for investigation after a few tries.

What should they configure?

A. Seek the subscription to a future time, to purge the stuck messages

B. Raise the ack deadline to its maximum of 600 seconds

C. Turn on exactly-once delivery for the subscription

D. Add a dead-letter topic, with an exponential backoff retry policy

◆ PAUSE AND DECIDE

What should they configure?

A Seek to the future

It purges every pending message, good ones included.

B A longer ack deadline

The message fails fast. A longer deadline changes nothing.

C Exactly-once delivery

It stops duplicates after acks. These messages are never acked.

D Dead-letter topic

Retries slow down, then bad messages move aside for investigation.

THE SIGNAL

'Keeps failing' plus 'set aside for investigation' is a dead-letter topic. 'Wasting capacity' adds backoff.

> Larkspur's payment service reads card payment events from a pull subscription in europe-west2. A payment must never be applied twice.
>
> Publishers sometimes retry after network errors. Each event carries a unique payment ID.

What is the right design?

A Enable exactly-once delivery, and still dedupe on the payment ID

B Enable exactly-once delivery on a push subscription to Cloud Run

C Rely on default delivery, because Pub/Sub never repeats a message

D Turn on message ordering, because ordering also removes duplicates

◆ PAUSE AND DECIDE

What is the right design?

A Exactly-once, deduped

No redelivery after acks, and the ID catches publisher retries.

B Exactly-once on push

Push subscriptions do not support exactly-once delivery.

C Default delivery

Default delivery is at least once. Duplicates are normal.

D Message ordering

Ordering controls sequence, not copies.

THE SIGNAL

'Never twice' plus 'publishers retry' means exactly-once delivery and a business-key check.

> Larkspur Energie runs Apache Kafka on-premises in Hamburg, with about 40 Kafka Streams applications.
>
> It wants to move them to Google Cloud with minimal code changes, while some stay on-premises for another year.

Where should the applications go?

A Rewrite every application for Pub/Sub, using ordering keys

B Move them to Managed Service for Apache Kafka

C Move them to Pub/Sub Lite, which also uses partitions

D Run self-managed Kafka on Compute Engine virtual machines

◆ PAUSE AND DECIDE

Where should the applications go?

A **Rewrite for Pub/Sub**

Forty rewrites, and two platforms during the transition.

B **Managed Kafka**

The Kafka API, so apps move as they are, and match on-premises.

C **Pub/Sub Lite**

Turned down on 18 March 2026. It is no longer an option.

D **Self-managed Kafka**

Same API, but brokers and upgrades become the team's job.

THE SIGNAL

Existing Kafka apps plus minimal change means Managed Kafka. Pub/Sub Lite is gone.

Module 11 in five lines

- Delivery is **at least once**. Design every subscriber to survive a second delivery.
- Ack deadline **10 to 600 seconds**. Missed deadlines look like duplicates.
- **Snapshot before deploys**. Seek to a time only if messages were retained.
- Backoff plus a **dead-letter topic** for poison messages. Exactly-once is **pull only**.
- Existing Kafka apps go to **Managed Kafka**. Pub/Sub Lite is gone.

COMING UP

Next: module 12, Dataflow and Beam, and how a pipeline really runs.